From Text Files to AI Agents

The Rise (and Transformation) of the Developer Environment

Brendan Shea, PhD

Programming and Problem Solving • Rochester Community and Technical College

Introduction

When you open VS Code today, you're greeted with syntax highlighting, instant error underlining, auto-complete suggestions, an integrated terminal, and a sidebar full of file management tools—all before you've typed a single character. It's easy to take for granted. But this sophisticated environment represents decades of innovation. And right now, it's undergoing its most radical transformation yet.

For most of computing history, writing code meant working directly with plain text files and running commands in a terminal. Then came the **Integrated Development Environment** (IDE)—software designed to put everything a programmer needs in one place. IDEs became so dominant that most programmers today have never worked without one.

But the rise of **AI coding assistants** and **agent interfaces**—tools where you describe what you want and an AI writes, edits, and runs the code—is raising a new question: what is the IDE *for*, and what will replace it? This case study traces that arc from the command line to the agent, and explains what it means for you as a beginning programmer.

Three Eras of Developer Tooling

Era 1 — The Text File Era (1960s–1980s): Programmers edit plain text, compile manually, and debug with print statements. The terminal is the entire environment.
Era 2 — The IDE Era (1990s–2010s): All-in-one applications bundle editing, compilation, debugging, and project management. The programmer is in control; the IDE assists.
Era 3 — The Agent Era (2020s–present): AI systems can read, write, and run code autonomously. The programmer increasingly acts as a director, reviewer, and guide—rather than the sole author.
None of these eras has ended. Text editors still exist. IDEs are thriving. And agents are just getting started.

1 Era 1: Plain Text and the Command Line

1.1 What "Writing Code" Once Meant

In the early decades of computing—the 1960s, 70s, and 80s—writing a program meant creating a plain text file and then *manually* running a series of commands to turn that text into something the computer could execute. There was no graphical interface, no color-coded text, no built-in spell-check for your syntax. Just you, a terminal (or even a physical punch-card machine), and a blinking cursor.

The Punch Card Era

Before video terminals became common, many programmers wrote code on physical punch cards—stiff paper cards where holes represented characters. A program might be a stack of hundreds of cards, each representing one line of code. If you dropped the stack and shuffled the order, you had to sort them by hand.

By the late 1970s, video terminals replaced punch cards, and programmers typed directly into text editors like `ed` (1969) and later `vi` (1976). These tools are still available on almost every Unix and Linux system today—and still used by experienced developers.

The workflow of a 1980s programmer looked something like this:

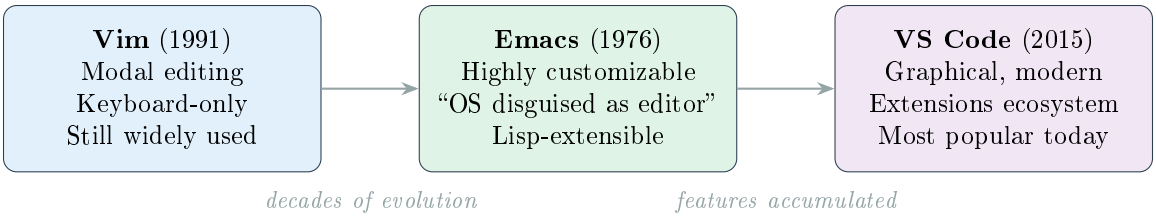
A Typical 1980s Workflow

```
# 1. Write source code in a text editor
vi HelloWorld.java

# 2. Save the file and exit the editor

# 3. Compile manually
javac HelloWorld.java

# 4. If there are errors, read them in the terminal,
#    go back to the editor, find the line, fix it, repeat

# 5. Run the program
java HelloWorld

# 6. If the output is wrong, add print statements to debug
```

This process—write, compile, read errors, fix, repeat—is called the **edit-compile-run cycle**. It's still the fundamental loop of programming today. The tools around it have changed enormously; the loop itself has not.

1.2 The Text Editors That Survived

Two text editors from this era are worth knowing about because they're still actively used and you'll encounter them on servers, in tutorials, and in job interviews.



Vim uses a **modal editing** model: the editor has different modes for inserting text, navigating, and running commands. It looks confusing at first (*how do I quit this thing?*) but its keyboard shortcuts let experienced users edit text very fast without ever touching a mouse. On any Linux server you'll ever access—a cloud virtual machine, a production system, a Raspberry Pi—Vim will be there.

Practical Tip: Learning Basic Vim

You don't need to master Vim, but knowing the basics will save you in situations where it's the only option (like editing a file on a remote server). Key commands:

- `i` — enter Insert mode (start typing)
- `Esc` — return to Normal mode
- `:w` — save the file
- `:q` — quit (use `:wq` to save and quit, `:q!` to quit without saving)
- `/word` — search for "word"

Run `vimtutor` in any terminal for a built-in 30-minute interactive lesson.

1.3 What Was Missing

Working with plain text editors and manual compilation worked, but it was slow and error-prone. Every typo was discovered only after compilation. Navigating large projects meant remembering file names. Debugging required inserting `System.out.println()` calls everywhere and recompiling. Refactoring—renaming a variable used in twenty files—was a manual, error-prone process.

Programmers knew things could be better. The tools that would emerge to solve these problems would reshape the profession.

2 Era 2: The Rise of the Integrated Development Environment

2.1 What an IDE Actually Is

An **Integrated Development Environment** combines, in a single application, the tools a programmer needs to write, run, and debug software. The word "integrated" is key: instead of separately invoking a text editor, a compiler, a debugger, and a file manager from the command line, all of these live together in one interface.

Early IDEs: Turbo Pascal and Visual Basic

The first widely successful IDEs appeared in the late 1980s and early 1990s. **Turbo Pascal** (Borland, 1983) was a revelation: you could write code, compile it, and run it from the same screen, and compilation was so fast it happened almost instantly. Many programmers encountered it as their first taste of an integrated environment.

Visual Basic (Microsoft, 1991) went further, adding a visual form designer: you could drag and drop buttons and text fields onto a window, and Visual Basic would generate the code for you. For beginners, seeing an application appear on screen with minimal code was genuinely exciting—and it introduced millions of people to programming.

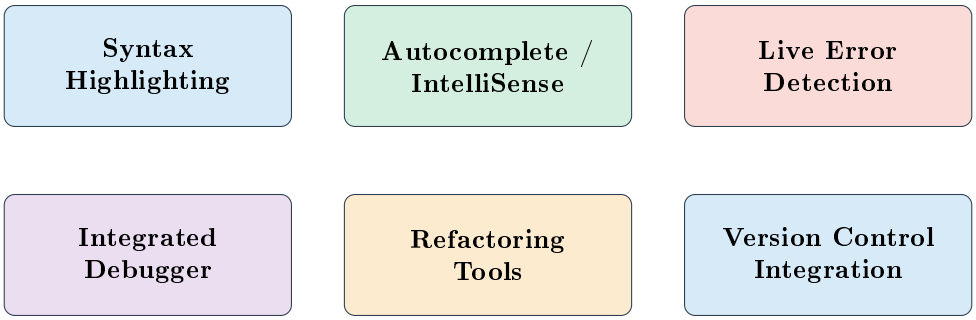
By the 2000s, two IDEs dominated:

- **Eclipse** (IBM, open-sourced 2001): A free, extensible IDE for Java that became the standard in enterprise development and universities. Heavy but powerful.
- **IntelliJ IDEA** (JetBrains, 2001): A commercial IDE with exceptional Java support. IntelliJ's "smart" features—deep code analysis, powerful refactoring—set a new standard for what an IDE could do.

Today, **Visual Studio Code** (Microsoft, 2015) is the most widely used editor in the world. It occupies a middle ground: lighter than Eclipse or IntelliJ, richer than plain text editors, and extensible through thousands of community-built plugins.

2.2 The Features That Changed Everything

What do IDEs provide that plain text editors don't? Several features that, once you've used them, feel indispensable:



Syntax highlighting colors keywords, strings, and variables differently, making code far easier to read at a glance. This sounds minor but has a profound effect on comprehension—your eye immediately catches a mismatched quote or an unrecognized keyword.

Autocomplete (called IntelliSense in VS Code) suggests method names, variable names, and parameters as you type. For a beginning programmer, this is enormously helpful: you don't need to memorize every method name, just the first few characters. For an expert, it eliminates tedious typing.

What IntelliSense Does For You

```
// You type:
String message = "Hello, world!";
message.

// IntelliSense immediately shows a dropdown:
//    charAt(int index)
//    contains(CharSequence s)
//    indexOf(String str)
//    length()
//    substring(int beginIndex)
//    toUpperCase()
//    ... and 40 more methods

// You don't need to remember the exact name.
// You pick from the list.
```

Live error detection underlines mistakes as you type—before you even try to compile. A red squiggle under a missing semicolon or a misspelled variable name means you fix problems immediately rather than discovering them minutes later during compilation.

The integrated debugger is perhaps the most powerful feature. Instead of adding print statements to understand what your code is doing, you can set a **breakpoint**—a marker telling the debugger to pause execution at that line. When the program pauses, you can inspect every variable's value, step through code line by line, and understand exactly what is happening.

Practical Tip: Use the Debugger Instead of Print Statements

New programmers often debug by adding `System.out.println()` everywhere. This works, but it's slow and clutters your code. Learn to use the debugger in VS Code instead:

1. Click in the margin next to a line number to set a breakpoint (a red dot appears)
2. Run your program in "Debug" mode (F5 in VS Code)
3. When the program hits your breakpoint, it pauses
4. Check the "Variables" panel to see every variable's current value
5. Press F10 to step to the next line, F11 to step *into* a method call

This skill separates programmers who struggle with bugs from those who solve them quickly.

2.3 Version Control Integration

Modern IDEs also integrate with **version control** systems, particularly **Git**. Git tracks every change you make to your code over time, allowing you to:

- Revert to any previous version if you break something
- See exactly what changed between two versions
- Collaborate with others without overwriting each other's work
- Maintain multiple experimental versions ("branches") simultaneously

VS Code shows which lines have been added, changed, or deleted since your last Git commit, right in the editor margin. You can stage changes, write commit messages, and push to GitHub without leaving the editor.

Practical Tip: Commit Early and Often

Beginning programmers often make many changes before committing to Git—then have trouble remembering what they changed and why. The habit to build: commit small, logical units of work with clear messages.

- Bad message: "stuff"
- Bad message: "fixed it"
- Good message: "Fix off-by-one error in loop that counted array elements"
- Good message: "Add input validation to prevent negative grades"

Future you—trying to understand why the code is the way it is—will be grateful.

2.4 Why IDEs Won

The IDE model succeeded because it dramatically reduced the cognitive overhead of programming. You could focus on solving the problem at hand rather than managing the toolchain. Errors surfaced immediately. Navigation in large codebases became manageable. And the visual debugger made understanding runtime behavior vastly easier than staring at print output.

By the 2010s, the question wasn't "should I use an IDE?" but "which IDE?" The era of the plain text editor, for most programmers doing most work, was over.

3 Era 3: AI Enters the Editor

3.1 The First Wave: Autocomplete Gets Smarter

The IDEs of the 2010s were good, but their autocomplete was still fundamentally mechanical: it looked up names in the current codebase and standard library. It couldn't suggest that you needed a loop here, or that you probably wanted to handle the null case, or that there was a smarter way to solve this problem.

That changed in 2021 when GitHub released **GitHub Copilot**, powered by OpenAI's Codex model. Copilot had been trained on billions of lines of publicly available code, and it could do something unprecedented: given a comment describing what you wanted, it would generate plausible working code.

Copilot-Style AI Completion (2021)

```
// Calculate the average of a list of integers,
// ignoring negative values
public double averagePositive(List<Integer> numbers) {
    // <- cursor here; AI suggests the entire method body:

    return numbers.stream()
        .filter(n -> n > 0)
```

```
            .mapToInt(Integer::intValue)
            .average()
            .orElse(0.0);
    }
```

This was qualitatively different from traditional autocomplete. The AI wasn't looking up a method name—it was generating novel code based on understanding what the comment described.

From Codex to Copilot to Cursor

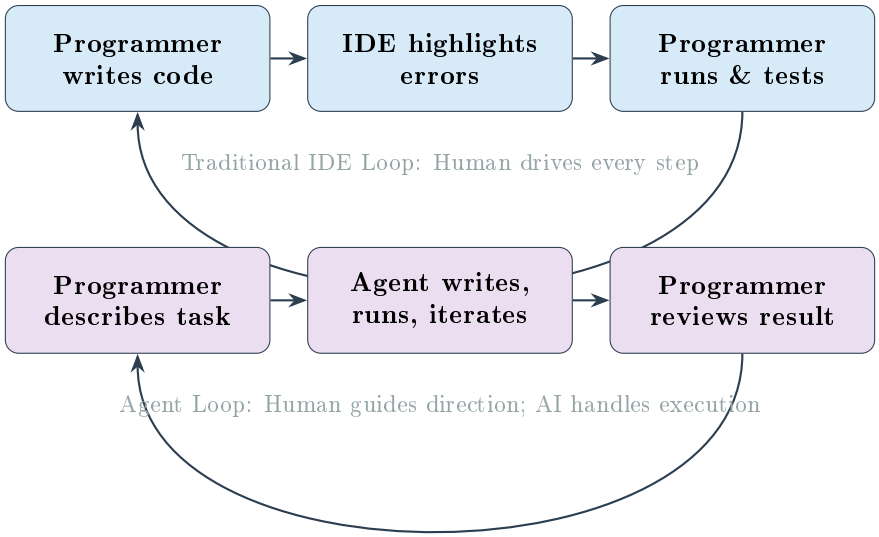
2021: GitHub Copilot launches as an IDE extension, offering AI-powered code completion inside VS Code and other editors.
2022–2023: The release of ChatGPT makes LLMs a mainstream phenomenon. Programmers start asking ChatGPT to explain errors, generate boilerplate, and suggest implementations—copy-pasting between the browser and their editor.
2023: **Cursor** launches—an editor *built around* AI. Rather than bolting AI onto VS Code, Cursor integrates AI deeply: you can select code and ask the AI to rewrite it, explain it, or fix it. The AI can see your entire codebase, not just the current file.
2024: Major IDEs (VS Code, JetBrains) add their own native AI features. The question shifts from "should I use AI?" to "which AI integration works best?"
2025: AI agents—systems that can autonomously run code, read files, search the web, and make changes across an entire codebase—move from research experiments to practical tools.

3.2 The Agent Interface: A Different Paradigm

AI code assistants inside editors still follow the IDE model: you, the programmer, are at the center, making every decision. The AI is a very capable autocomplete. But **AI coding agents** represent something more fundamental.

An agent doesn't just suggest the next line—it can:

- Read your entire codebase to understand context
- Write new files or edit existing ones
- Run your tests and interpret the results
- Search the web for documentation or examples
- Execute code, observe output, and fix errors it finds
- Perform multi-step tasks spanning many files and many minutes



Tools like **Claude Code**, **Devin**, **SWE-agent**, and **OpenAI Codex** (distinct from the original) allow a programmer to say things like: "Add input validation to all the form fields in this project," or "Find and fix the bug that causes the tests to fail," and watch the agent do it—reading files, writing code, running tests, correcting mistakes.

What Interacting With an Agent Looks Like

Programmer: "Our `StudentGrade` class doesn't handle the case where a student has no assignments. Add appropriate handling and write tests for it."
Agent: *[Reads `StudentGrade.java`, reads `StudentGradeTest.java`]*
"I see the issue—`calculateAverage()` divides by the list size without checking if it's empty. I'll fix this and add three new test cases."
[Writes the fix, writes the tests, runs the test suite]
"All 14 tests pass. Here's what I changed and why: [explanation]"
Programmer: "The logic looks right, but use 'Optional' for the return type instead of throwing an exception."
[Agent updates implementation and tests]

This isn't science fiction. It's how many professional programmers are working today.

4 What This Means for You as a Beginning Programmer

4.1 The Temptation and the Trap

For a beginning programmer, AI tools are seductive—and that's worth thinking carefully about. If you can describe what you want and an AI writes the code, why learn to write it yourself?

The honest answer is that this is a real and open question. But there are good reasons to invest in understanding code deeply, even in an age of AI assistance.

The Understanding Gap

AI tools work best when you can evaluate what they produce. When an AI generates code and it fails, you need to understand *why* it's wrong. When an AI produces working code that's inefficient or hard to maintain, you need to recognize that.

More subtly: AI tools make mistakes. Sometimes the mistakes are obvious (a syntax error); sometimes they're subtle (a logic error that only triggers for edge cases; a security vulnerability that looks fine). A programmer who doesn't understand the underlying concepts can't catch these mistakes.

Beginning programmers who rely too heavily on AI tend to get stuck on problems the AI handles poorly: novel problems, unusual requirements, debugging AI-generated code that's broken in non-obvious ways.

The analogy: knowing how to use a calculator is valuable, but students who skip learning arithmetic often struggle with algebra, because they lack a feel for whether answers make sense. Programming is similar. AI tools amplify your abilities—but if your abilities are thin, there's little to amplify.

4.2 Practical Tips for Using AI Tools as a Beginner

AI tools are genuinely useful for learning—if used thoughtfully.

Using AI to Learn Rather Than Just Get Answers

- **Ask for explanations, not just code.** Instead of "Write a method to sort this list," try "Write a method to sort this list *and explain each line*." Reading the explanation teaches you the pattern.
- **Predict before you accept.** When AI suggests a completion, pause and think: what do I expect this code to do? Then accept it, run it, and check your prediction. This builds the habit of reading code critically.
- **Type it yourself first.** Try solving the problem before asking AI for help. Even if you only get partway, you'll understand the AI's answer better for having struggled.
- **Use AI to debug, not to avoid understanding.** If your code is broken, describe the bug to an AI and ask it to help you understand what's wrong—not just to fix it for you.
- **Review AI output like code review.** Treat AI-generated code the way a professional treats a colleague's code: read it, understand it, and question anything that seems wrong or unclear.

4.3 VS Code Is Not Dead

You might come away from this case study thinking: if agents are the future, should I even bother learning VS Code? The answer is unambiguously yes—for several reasons.

Why VS Code Still Matters

Agents live inside IDEs. GitHub Copilot, Cursor, and most other AI tools are either VS Code extensions or editors built on VS Code's codebase. Learning VS Code means learning the environment where AI tools live.

You still write code. Even with AI assistance, programmers spend significant time reading, understanding, and making targeted edits to code. An IDE makes all of this faster.

The debugger is irreplaceable. No AI tool makes the debugger less valuable. When something goes wrong—and things will go wrong—being able to pause execution and inspect state is essential.

Skills transfer. VS Code fluency—keyboard shortcuts, extensions, Git integration, the terminal—transfers to professional settings. Knowing your tools is a professional baseline.

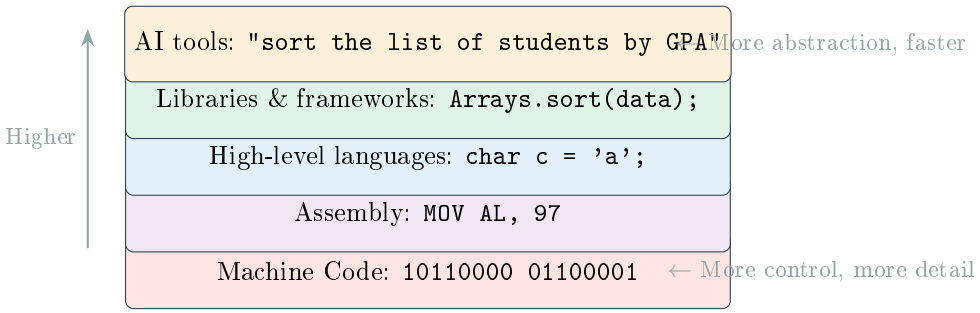
The agent era is early. AI agents are impressive but imperfect. They make mistakes, get confused about requirements, and need human oversight. For the foreseeable future, programmers who understand the code will be more effective than those who don't.

5 The Bigger Picture: What is the Programmer's Role?

5.1 Tools Always Change the Work

The history of programming tools is a history of raising the level of abstraction. Early programmers wrote in binary machine code—literally sequences of 0s and 1s. Assembly language let them write `ADD R1, R2` instead. High-level languages like Fortran and COBOL let them write something resembling math. Object-oriented languages let them model concepts directly. And now AI tools let them describe intent in natural language.

Each step up the ladder of abstraction made programmers more productive—and shifted the valuable skill from *knowing the details* toward *knowing what to build and whether it's right.*



The programmer's role shifts with each level. At the machine-code level, you must understand every bit. At the AI level, you must understand what you want to build, how to evaluate whether it's built correctly, and how to guide the AI when it goes wrong.

5.2 New Skills for the Agent Era

If AI tools handle more of the typing and routine problem-solving, which skills become more valuable?

- **Problem decomposition:** Breaking a large, vague task into small, concrete steps that an AI (or a human) can execute. This is a skill that must be learned and practiced.

- **Code reading and review:** Understanding unfamiliar code quickly—knowing what a function does, spotting subtle bugs, recognizing bad patterns. This is harder to fake with AI assistance.
- **System design:** Deciding how pieces fit together—which classes to create, how components communicate, what data model to use. AI can suggest, but humans must judge.
- **Testing and verification:** Knowing what "correct" means and how to verify it. Writing good tests remains a high-value skill even if AI writes the implementation.
- **Communication:** Describing requirements clearly—to teammates, to AI agents, and to stakeholders. Vague requirements produce vague (and wrong) results.

Will AI Replace Programmers?

This question comes up constantly, and the honest answer is: partially, and unevenly.
Routine, well-defined coding tasks—writing boilerplate, translating specifications into standard patterns, fixing common bugs—are increasingly automatable. Some entry-level roles that consisted primarily of these tasks may shrink.
But programming is not just writing code. It involves understanding users' real needs (which they often can't fully articulate), making design decisions with long-term consequences, evaluating tradeoffs, and ensuring correctness. These remain stubbornly human-centered activities.
More likely than replacement: the *kind* of programmer work changes. Professionals who learn to work effectively with AI tools—directing, reviewing, and correcting AI-generated code—will be more productive than those who don't. The skill of working *with* AI is itself becoming a professional competency.

6 Getting Started: Your Recommended Setup

If you're just beginning, here's a practical path that prepares you for both the current landscape and where things are heading.

Your Recommended Development Setup

Step 1: Learn VS Code. Download it free at `code.visualstudio.com`. Install the Java Extension Pack. Spend time learning:

- Basic keyboard shortcuts (Ctrl+P to find files, Ctrl+Shift+P for commands)
- How to set breakpoints and use the debugger
- The integrated terminal (`Ctrl+‘`)

Step 2: Learn Git basics. Git is non-negotiable in professional software development. Learn to `init`, `add`, `commit`, `push`, and `pull`. Set up a free GitHub account and push your projects there.
Step 3: Try an AI assistant—carefully. GitHub Copilot offers a free tier for students. Install it in VS Code and experiment. Use it to explain code you don't understand, suggest completions for patterns you half-remember, and generate test cases. Watch it make mistakes. Ask it to explain its reasoning.

Step 4: Build projects you care about. The best way to learn any tool is to use it for a real goal. Build something you actually want: a grade calculator, a simple game, a tool that solves a problem you have. Tools make sense in context.

Discussion Questions

Discussion Questions

1. **The Learning Paradox:** AI tools can write code that beginning programmers can't yet write themselves. Is this a good thing—a way to see correct solutions and learn from them—or a bad thing, because it bypasses the struggle that produces deep understanding? Where do you draw the line in your own learning?
2. **The Abstraction Ladder:** Each step up the abstraction ladder (machine code → assembly → high-level languages → AI tools) made some skills obsolete while making others more valuable. What skills are we in danger of losing as AI writing code becomes common? Are there skills from earlier eras that we've already lost—and do we miss them?
3. **The Review Problem:** A core argument for learning to code is that you need to review and verify AI-generated code. But if AI gets good enough, will human review remain meaningful? Could an AI review another AI's code? What would that mean for the human role?
4. **The IDE as Interface:** The IDE gave programmers a powerful environment for working with code. The agent interface gives programmers a way to *direct* code without necessarily writing it. What kinds of people might be better served by the agent interface than the IDE? Does this expand or contract who can participate in software development?
5. **Your Own Practice:** How do you currently use (or plan to use) AI tools in your own programming? What guardrails or habits do you think you need to develop to make sure these tools enhance rather than substitute for your own understanding?

Key Terms

Glossary of Key Terms

Agent	An AI system that can autonomously perform multi-step tasks: reading files, writing code, running programs, and fixing errors—without requiring a human to drive each step.
Autocomplete	An IDE feature that suggests completions for the code you are typing, based on the available methods, variables, and libraries.
Breakpoint	A marker placed in code that causes a debugger to pause execution at that line, allowing inspection of program state.
Debugger	A tool that lets programmers pause a running program, inspect variable values, and step through code line by line to find errors.

Edit-Compile-Run Cycle The fundamental loop of programming: write code, compile it, run it, observe the results, and repeat.

Git A version control system that tracks changes to code over time, enabling history, collaboration, and the ability to revert to earlier versions.

GitHub Copilot An AI-powered code assistant (by GitHub and OpenAI) that suggests code completions and generates entire functions from natural-language descriptions.

IDE Integrated Development Environment; an application combining a code editor, compiler, debugger, and other tools in a single interface.

IntelliSense Microsoft's name for the smart autocomplete feature in VS Code, which shows method signatures, documentation, and context-aware suggestions.

Modal Editing Vim's approach to text editing, where the editor has separate modes for inserting text and for navigating and manipulating it.

Syntax Highlighting Coloring different parts of code (keywords, strings, variables) to make structure immediately visible.

Version Control A system for tracking changes to files over time, allowing reversion, comparison, and collaboration.

VS Code Visual Studio Code; a free, extensible code editor by Microsoft, currently the most widely used editor among developers worldwide.

This case study is part of the Open Educational Resources for Programming and Problem Solving.
Licensed under Creative Commons Attribution 4.0 (CC BY 4.0).